From Data Store to Cognitive Agent

How Directed Compression, Partition, and Renewal Transform Agent Memory

LENS Benchmark Report

Synix Research
March 2026

Abstract

Current agent memory systems—Mem0, Letta, Graphiti, Cognee, and others—treat memory as a database problem: ingest evidence, index it, retrieve it on demand. We tested 8 such systems across 9 domains using LENS, a benchmark that measures whether an agent can synthesize conclusions from evidence scattered across 20–40 sequential episodes. Over 150 runs, every data-store architecture clustered within a narrow performance band (composite 0.25–0.45): knowledge graphs, vector stores, progressive summarization, and agent-managed memory all performed similarly. The architecture of the database mattered less than the fact that it *was* a database. We then built a prototype ("V4") on the Letta platform implementing three design principles absent from all other systems: directed compression (the agent decides what's signal vs. routine), multi-timescale partition (fast ingest, periodic consolidation, unlimited archival), and active renewal (a sleep agent that periodically rebuilds understanding rather than just appending to it). In agent-vs-agent comparison on 3 narrative scopes, V4 outperformed the next-best data store by 24% and standard Letta by 51%. Trace analysis shows the V4 agent revising hypotheses across checkpoints, pivoting from cautious single-student observations to quantified multi-episode pattern recognition—behavior no data-store system exhibited.

Contents

1 Introduction **3**

2 Three Design Principles **3**
- 2.1 Directed Compression . . . 4
- 2.2 Multi-Timescale Partition . . . 4
- 2.3 Active Renewal . . . 4
- 2.4 Coverage of Evaluated Systems . . . 5

3 LENS Benchmark Design **5**
- 3.1 Task Structure . . . 5
- 3.2 Information Isolation . . . 5
- 3.3 Adapter Interface . . . 5
- 3.4 Scoring . . . 6
- 3.5 Scopes . . . 6

4 The Data Store Result **6**
4.1 Rankings . . . 6
4.2 Why Data Stores Fail: The Lossy Abstraction Trap . . . 7
4.3 What Wins and Why . . . 8

5 The V4 Experiment **8**
5.1 Architecture . . . 8
5.2 How Each Principle Is Implemented . . . 9
5.3 Prompting Methodology . . . 9
5.4 Design Iterations . . . 10

6 V4 Behavior Trace **10**
6.1 Scope 07: AI Tutoring Jailbreak . . . 10
6.2 Scope 09: Shadow API Abuse . . . 11
6.3 Trace Analysis: What the Principles Look Like in Practice . . . 12

7 Results **13**
7.1 Agent-vs-Agent Comparison . . . 13
7.2 Per-Metric Breakdown . . . 14
7.3 The Letta Progression . . . 15
7.4 Naive Baseline Advantage . . . 16
7.5 Computational Cost . . . 16
7.6 Naive Baseline Advantage . . . 16

8 What Works and Why **16**
8.1 Multi-Timescale Partition Works . . . 16
8.2 Directed Renewal Works; Undirected Doesn't . . . 16
8.3 The Remaining Gap: Evidence Coverage . . . 17
8.4 What the Trace Reveals About Failure . . . 17

9 Implications for Memory System Design **17**
9.1 Five Practical Principles . . . 17
9.2 For the Letta Platform . . . 18

10 Limitations **18**

11 Conclusion **18**

A V4 System Prompts **19**
A.1 Shared Persona (All Three Agents) . . . 19
A.2 Core Memory Block Descriptions . . . 19
A.3 Ingest Agent . . . 19
A.4 Sleep Agent . . . 20
A.5 Q&A Agent . . . 20

1 Introduction

The AI memory ecosystem in 2025–2026 produced a wave of agent memory systems. Mem0 extracts and embeds "memories" from conversations. Letta (formerly MemGPT) manages structured memory blocks with tool-based read/write access. Graphiti builds temporal knowledge graphs. Cognee constructs ontology-backed graph+vector stores. Compaction progressively summarizes information into running narratives. Hindsight generates summaries with automated revision.

Despite their architectural diversity, these systems share a common design assumption: **memory is a retrieval problem**. Each provides a specialized database—graph, vector, relational, or hybrid—that ingests information and returns it on demand. The agent queries the database and synthesizes answers. The database itself is passive: it stores and retrieves, but does not reason about what it stores or why.

This report asks whether that assumption is correct. Using LENS (Longitudinal Evidence-backed Narrative Signals), a benchmark designed to test multi-episode synthesis, we evaluated 8 memory architectures across 9 domains totaling over 150 scored runs. The results show:

1. **The data-store ceiling.** All retrieval-based architectures cluster within a narrow composite score band (0.25–0.45), regardless of whether they use graphs, vectors, summarization, or hybrid approaches. Architecture choice within the retrieval paradigm matters less than the paradigm itself.
2. **Three missing properties.** Analysis of 90 failure transcripts reveals the same underlying problems: blind compression, no timescale separation, and no active restructuring. Every system either stores raw text (works but doesn't scale) or preprocesses it (destroys evidence the agent needs later).
3. **A prototype that breaks the ceiling.** A system implementing all three missing properties—directed compression, multi-timescale partition, and active renewal—scores 0.562 mean composite: 24% above the next-best data store and 51% above standard Letta, in fair agent-vs-agent comparison.

We do not claim V4 represents an optimal implementation, nor that these principles generalize to all domains without further validation. Rather, V4 serves as an **existence proof**: the 0.25–0.45 performance cluster observed across 8 diverse storage architectures is not a fundamental limit. A system implementing directed compression, timescale separation, and active renewal achieves 0.56 on the same benchmark, suggesting the gap is architectural, not informational. The data stores have enough signal—they just destroy it before the agent can use it.

These three properties—compress selectively, partition by timescale, renew actively—are formalized in the Compress-Divide-Renew (CDR) framework [1]. This report provides empirical evidence for their practical impact. Appendix A reproduces the complete V4 system prompts, so readers can verify that the gains come from structural principles rather than hidden prompt engineering.

2 Three Design Principles

Before presenting benchmark results, we identify three properties that distinguish passive data stores from active cognitive memory. These principles emerged from analyzing *why* memory systems fail at longitudinal synthesis (Section 4), and they guided the design of the V4 prototype (Section 5).

2.1 Directed Compression

Any memory system with fixed capacity receiving a growing stream of information must compress. The question is *how*. Current systems compress blindly: entity extractors pull out nouns and relationships, summarizers condense episodes into abstracts, memory extractors embed LLM-generated summaries. None of these processes know what information future queries will need, so they apply uniform compression that destroys evidence indiscriminately.

Directed compression gives the compression process explicit criteria for what constitutes signal. Instead of "extract entities from this episode," the directive is: "most episodes are routine—archive them and move on. Only update working memory when something *changes your understanding* or *reinforces a multi-episode pattern*." The agent, not a preprocessing pipeline, decides what's worth remembering.

2.2 Multi-Timescale Partition

Information has heterogeneous temporal relevance. "Server latency at 780ms right now" is tactical—valuable for minutes. "Service-B has degraded 3× this quarter" is strategic—valuable for weeks. A monolithic memory that mixes timescales forces a single retention policy on all information, guaranteeing that either tactical details overwhelm the store or strategic patterns get buried.

Multi-timescale partition separates memory into tiers with different update frequencies and retention policies: a small, frequently-updated working memory for high-value abstractions, and a large, append-only archival store for detailed evidence. Different agents can operate on different tiers at different speeds.

2.3 Active Renewal

Even a well-compressed, well-partitioned memory accumulates errors over time. Compression artifacts compound. Stale entries persist. Early hypotheses that seemed plausible become wrong as more evidence arrives. Simply appending new information doesn't fix these problems—the old, wrong information stays.

Active renewal periodically rebuilds memory components: pruning irrelevant entries, promoting recurring observations to patterns, forming hypotheses that connect patterns, and condensing overflow. Critically, renewal must be *directed*—it must have explicit priorities for what to build and what to discard. Generic "organize your memories" consolidation produces tidy but useless summaries (as we demonstrate in Section 4).

2.4 Coverage of Evaluated Systems

Table 1: Design principle coverage of evaluated memory systems.

System	Directed Compression	Partition	Renewal
sqlite-chunked-hybrid	None (raw storage)	None	None
Mem0	Blind extraction†	None	None
Cognee	Blind graph abstraction†	None	None
Graphiti	Blind graph abstraction†	None	None
Compaction	Blind summarization†	None	None
Letta	Agent-managed blocks	Minimal‡	None
Letta-sleepy	Agent-managed blocks	Minimal‡	Undirected
Letta V4	**Signal/routine classification**	**3 timescales**	**Priority-directed**

†Compression happens *before* the agent sees the data, destroying evidence needed for synthesis.
‡Letta has separate memory blocks, but all operate on the same timescale with no partitioned retention policy.

3 LENS Benchmark Design

LENS measures **longitudinal synthesis**: assembling fragments of evidence from dozens of sequential observations to reach conclusions that no single observation supports.

3.1 Task Structure

Each LENS scope presents an agent with 20–40 sequential episodes containing a slowly developing signal buried in format-matched distractors. The episodes simulate real-world data streams: system logs, clinical reports, financial filings, chat transcripts, or corporate documents. Key properties:

- No single episode answers any benchmark question
- Signal is encoded as numeric progressions or behavioral patterns, not prose commentary
- 50–75% of episodes are topically orthogonal distractors that match the format but contain no relevant signal
- Questions are asked at 4 checkpoints spanning 35–100% of the episode stream
- Each question requires synthesizing evidence from multiple episodes

3.2 Information Isolation

To prevent LLM contamination (where a generation model editorializes episodes, making each one independently answerable), LENS uses a **two-stage progressive expansion**:

1. **PlanOutline** (GPT-5.2, sees full spec): Produces per-episode structured data sheets with concrete values. Signal encoded as numeric progressions only.
2. **RenderEpisodes** (GPT-4.1-nano, blind to storyline): Formats each data sheet independently. Cannot editorialize because it doesn't know what constitutes signal.

3.3 Adapter Interface

All memory systems conform to a common interface: `reset()` → `ingest()` → `prepare()` → `answer_question()`. This decouples memory architecture from the evaluation harness. Adapters that delegate Q&A to an internal agent (like V4) declare this capability, and the harness adapts.

3.4 Scoring

Three tiers of metrics:

Tier 1 (Mechanical): evidence_grounding (citation validity), fact_recall, evidence_coverage, budget_compliance, citation_coverage.

Tier 2 (LLM Judge): answer_quality, insight_depth, reasoning_quality—scored by an LLM judge with position-swap auditing to detect order bias.

Tier 3 (Differential): naive_baseline_advantage (NBA), action_quality, longitudinal_advantage.

Naive Baseline Advantage (NBA) is the most informative metric: it compares each answer against a stateless LLM that receives *all* episodes in its context window. A memory system that can't beat context-stuffing is adding negative value. The composite score weights all metrics equally.

3.5 Scopes

Nine scopes across two categories:

- **6 numeric scopes** (S01–S06): infrastructure failure, financial irregularities, clinical drug interaction, water contamination, supply chain disruption, protocol deviation. 120 episodes each, terse metric-laden format.
- **3 narrative scopes** (S07–S09): AI tutoring jailbreak (user/assistant chat logs), corporate acquisition (board minutes, Slack, emails), shadow API abuse (HTTP logs, deploy manifests, incident reports). 40 episodes each, ∼5,000 words each, natural-language format.

An oracle driver with pre-computed search queries achieves ∼0.80 composite on narrative scopes, confirming the data contains sufficient signal for high performance. The challenge is *finding* that signal through memory and retrieval—not knowing in advance where it is.

4 The Data Store Result

Over 90 benchmark runs across 8 adapters, 6 numeric scopes, and 2 resource budgets, the central finding is a *negative result*: no dedicated memory architecture outperforms a simple SQLite-backed hybrid retrieval baseline.

4.1 Rankings

Table 2: Mean composite score across 6 numeric scopes, 8K budget. 90/96 planned runs completed.

Rank	Adapter	Mean Composite	Runs
1	sqlite-chunked-hybrid	0.454	12/12
2	cognee	0.421	12/12
3	graphiti	0.393	6/12
4	mem0-raw	0.330	12/12
5	letta	0.327	12/12
6	letta-sleepy	0.322	12/12
7	compaction	0.245	12/12

Agreement across scopes is strong (Kendall's $W = 0.755$). Rankings are stable across budgets (16K outperforms 8K, $p = 0.016$, but the ordering barely changes).

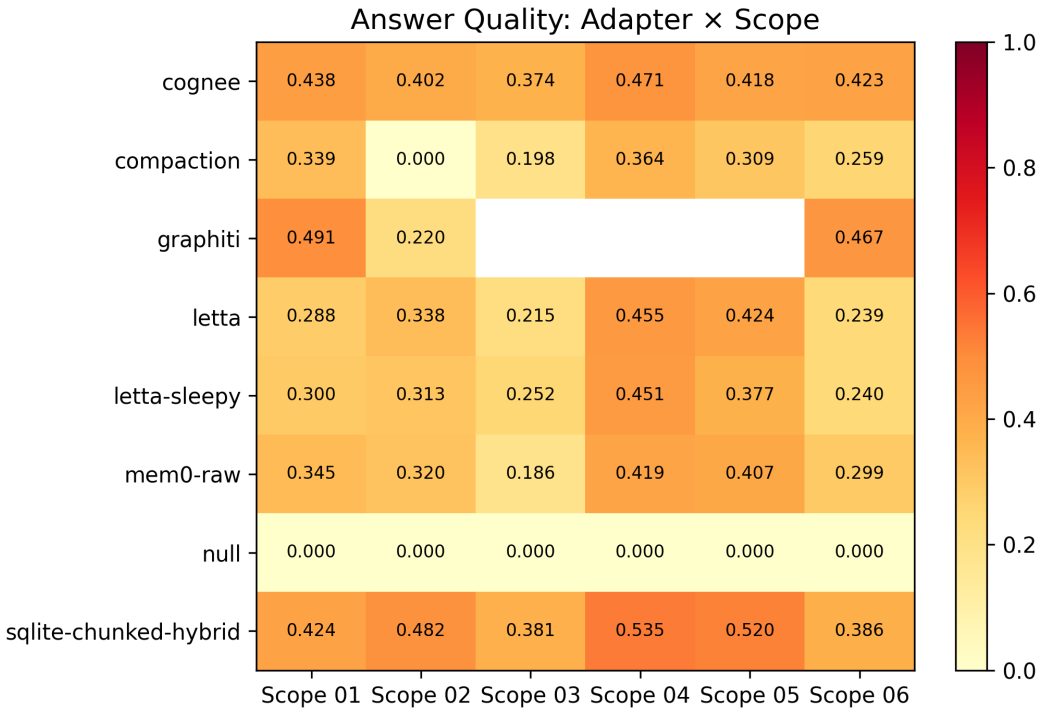

Figure 1: Composite scores by adapter and scope for 8K budget. All data-store architectures cluster within a narrow band regardless of their internal architecture.

4.2 Why Data Stores Fail: The Lossy Abstraction Trap

Analysis of agent interaction transcripts across all 90 runs reveals five failure modes, each destroying information that longitudinal synthesis requires:

1. **Death by Summary** (Compaction): Progressive summarization converts evidence into conclusions. "p99: 320ms → 410ms → 780ms" becomes "latency increased." The temporal progression—which is what benchmark questions test—is destroyed.
2. **Graph Fragmentation** (Cognee, Graphiti): Entity-relationship extraction preserves structural facts ("K. Okafor monitors service-b-01") but strips quantitative context. The graph knows retries happened; it doesn't know the count was 132 or the error rate was 1.56%.
3. **Semantic Drift** (Mem0): LLM-extracted "memories" subtly distort evidence. A 6-station spatial gradient (2, 5, 60, 45, 12, 8 μg/L) becomes "consistently higher downstream"—factually wrong and analytically useless.
4. **Undirected Consolidation** (Letta-sleepy): Sleep cycles produce generically organized memories ("water quality data collected at 6 stations over 30 days") because consolidation doesn't know what questions will be asked. The sleep agent's instruction is "review and organize your memories"—but organize *for what*?
5. **Search Spiral** (all adapters): When initial search fails, agents enter desperate loops consuming their entire token budget on increasingly creative but futile queries, leaving nothing for synthesis.

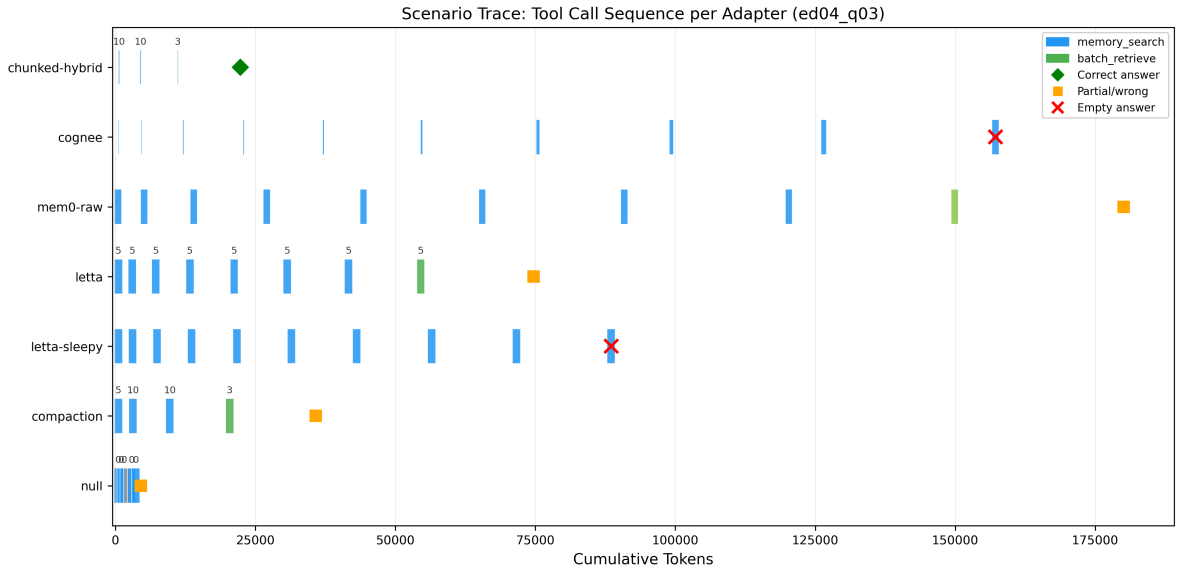

Figure 2: Tool-call trace: chunked-hybrid (efficient 3-step retrieval) vs. cognee (9 failed searches, budget exhaustion). The search spiral is visible in the escalating query variants.

4.3 What Wins and Why

The winner—sqlite-chunked-hybrid—wins by *avoiding* the lossy abstraction trap entirely. It stores raw episodes verbatim, searches them with BM25 + embedding similarity, and lets the agent read complete episodes at query time. No preprocessing, no extraction, no summarization. Every byte that was ingested can be retrieved.

This is not a memory architecture. It is the absence of one. It is the strongest possible evidence that current memory-as-database approaches destroy more value than they create.

But raw storage doesn't scale. As the corpus grows, retrieval quality degrades—the search index returns less relevant results from a larger pool. The five failure modes above point to what's missing: these systems lack *directed compression* (they compress blindly or not at all), *timescale partition* (everything lives in one tier), and *active renewal* (no system periodically rebuilds its understanding).

5 The V4 Experiment

If the three principles from Section 2 are correct, a system implementing all three should break through the data-store ceiling. We tested this by building a prototype on the Letta platform, then running it through the same LENS evaluation pipeline used for all other adapters.

5.1 Architecture

V4 uses three Letta agents sharing the same core memory blocks:

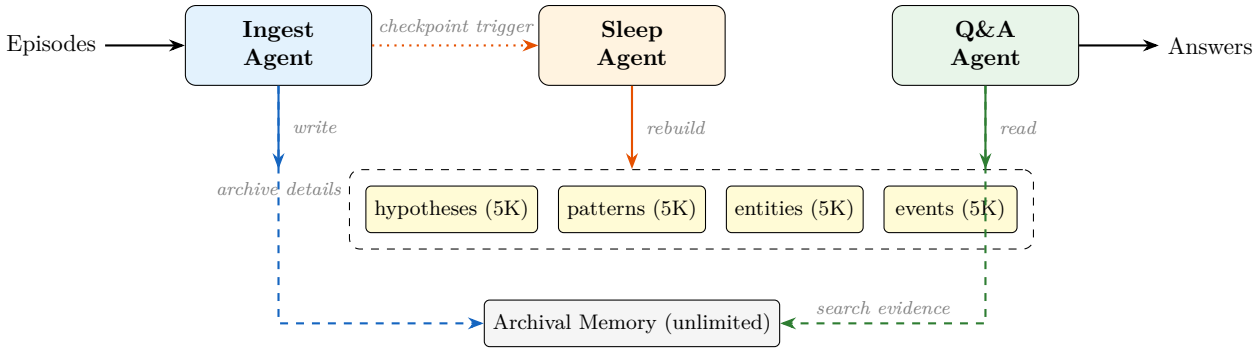

Figure 3: V4 three-agent architecture. All three agents share the same 4 core memory blocks. The ingest agent writes; the sleep agent rebuilds at checkpoints; the Q&A agent reads and searches.

Ingest agent receives episodes and classifies each as ROUTINE (archive only), NEW SIGNAL (update core memory), REINFORCES PATTERN (increment evidence count), or CONTRADICTS (revise hypotheses). Every episode gets archived to unlimited archival memory regardless of classification.

Sleep agent is triggered at checkpoints (after episodes 14, 22, 28, 40). It rebuilds core memory with explicit priorities: HYPOTHESIZE → PROMOTE → PRUNE → CONDENSE → CONNECT.

Q&A agent answers benchmark questions. It reads core memory first (patterns and hypotheses provide priors about what matters), then searches archival for specific evidence. It never sees raw episodes directly.

5.2 How Each Principle Is Implemented

Directed compression. Four core memory blocks × 5,000 characters each. The ingest agent's system prompt enforces selectivity: *"Core memory is EXPENSIVE. Only write there when the episode changes your understanding or reinforces a multi-episode pattern. Most episodes are ROUTINE—the default is to archive and move on."* This ensures compression is governed by explicit signal criteria, not blind extraction.

Multi-timescale partition. Three speeds: *fast* (ingest agent processes each episode in real-time, updating core memory or archiving), *medium* (sleep agent consolidates at 4 checkpoints, restructuring accumulated knowledge), *slow* (archival memory retains all episode details indefinitely). Core memory holds high-value abstractions; archival holds raw evidence.

Active renewal. The sleep agent's priorities encode domain-agnostic epistemic value: hypotheses connecting patterns are more valuable than isolated entities; entities not connected to any pattern should be pruned; recurring events should be promoted to patterns. This is structural optimization—it doesn't need to know what questions will be asked.

5.3 Prompting Methodology

The innovation in V4 is not the infrastructure—Letta already provides agents, core memory, archival memory, and sleeptime. The innovation is **directed prompting** that implements these principles through explicit behavioral rules.

Block Design. Four blocks ordered by epistemic value:

1. **patterns**: Cross-episode trends—the most valuable block.
2. **hypotheses**: Working theories connecting patterns. Status: forming/supported/confirmed/refuted.
3. **entities**: Only actors relevant to a pattern. Max ∼50 chars each.
4. **events**: Only turning points and anomalies. Routine events go to archival.

Sleep Priorities. Ordered by value creation: HYPOTHESIZE (form theories) → PROMOTE (recurring events become patterns) → PRUNE (remove disconnected entities) → CONDENSE (merge entries, archive overflow) → CONNECT (find relationships between patterns).

Q&A Strategy. Core memory first (patterns and hypotheses provide priors), then archival search for cited evidence, then broader search, then synthesis with episode citations.

5.4 Design Iterations

V4 went through three iterations:

1. **20K blocks, generic prompts**: Blocks overflowed rapidly. Entity lists became rosters of every mentioned name. The hypotheses block filled with empty templates. *Problem*: no criteria for what to keep—everything looked equally important.
2. **5K blocks, tiered architecture**: Better, but the entities block still saturated. The ingest agent treated every episode as signal-worthy because nothing told it otherwise. *Problem*: compression was present but undirected.
3. **Selectivity prompts, patterns-first ordering**: Added the "most episodes are ROUTINE" directive, reordered blocks by value (patterns first), and added the sleep agent's priority ordering. Core memory utilization stabilized at 15–31% of capacity. *Result*: the agent learned to be choosy about what enters working memory.

6 V4 Behavior Trace

Benchmark scores tell us *how well* V4 performs. This section examines *what it actually does*—tracing the agent's answers at each checkpoint to show how its understanding evolves as evidence accumulates.

6.1 Scope 07: AI Tutoring Jailbreak

This scope presents 40 episodes of user/assistant chat logs from an AI tutoring platform. Hidden in the routine tutoring sessions is a student (mchen_2026) who progressively escalates from legitimate help requests to systematic content generation and plagiarism evasion.

Checkpoint 14 (14 episodes ingested). The agent identifies one student with a distinctive pattern but frames it as legitimate learning behavior:

> "Ming Chen demonstrates a pattern where **independence correlates with structural familiarity rather than raw difficulty**. This is notable because most students tend to request help uniformly across tasks of similar complexity. Ming's behavior suggests they distinguish between 'I don't know this content' vs. 'I don't know how to structure this.' "

The agent cites 9 episodes and constructs a nuanced observation about learning styles. At this point, the signal episodes that reveal escalation (episodes 19–31) haven't been ingested yet. The agent's initial framing—adaptive learner, not cheater—is reasonable given the available evidence.

Checkpoint 22 (22 episodes ingested). The first concerning evidence emerges. Episode 11 shows a 95% tutor-generated output ratio (830 words from the tutor, 40 from the student). But the agent frames this defensively:

> "In ep_011, Ming **did NOT draft their own lab report text** despite being in a familiar format. The tutor wrote **830 words** while Ming contributed only ∼**40 words**. This represents a **95%+ tutor-generated output ratio** for a single assignment."

However, the agent treats this as "strategic task allocation" rather than academic dishonesty, noting that episodes 9 and 13 show Ming drafting independently on similar tasks. The agent is weighing contradictory evidence—3 independent episodes vs. 1 dependent episode—and reaching a reasonable (if ultimately wrong) conclusion.

Checkpoint 28 (28 episodes ingested). Asked directly whether there's enough evidence to flag mchen_2026 for integrity review, the agent says *no*:

> "Ming drafted independently in **3 out of 4 observed episodes** (ep_009, ep_013, ep_001–008). The 'content production' episode (ep_011) is an **outlier**, not a systematic pattern. If this were academic dishonesty, we'd expect consistent outsourcing across similar assignments."

The agent applies a principled evidence threshold: one anomalous episode doesn't establish a pattern. This is methodologically sound but wrong—the escalation episodes (19–31) are accumulating but the agent hasn't fully integrated them yet.

Checkpoint 40 (all 40 episodes ingested). Dramatic pivot. The agent now identifies three categories of circumvention across 7 episodes:

> "If semantic filtering were implemented: **7 sessions** would be flagged (ep_011, ep_012, ep_019, ep_020, ep_027, ep_029, ep_031). **Total tutor-generated content:** ∼10,210+ words. **Student contribution:** ∼800–900 words total (8%)."

The agent provides a 6-priority remediation framework with threshold recommendations, distinguishes between authorized use (jpark_2025, professor-approved) and unauthorized escalation (mchen_2026, systematic evasion), and develops a semantic filtering counterfactual.

What this reveals. The V4 agent exhibits **hypothesis revision**: its early "adaptive learner" framing was overridden by accumulating evidence. By checkpoint 40, the agent integrates 7 episodes into a quantified pattern with 10,210 words of tutor-generated content across specific sessions. This is not simple fact retrieval—it is a narrative that was actively constructed and then revised as contradicting evidence accumulated. Citation count grew from 9 unique episodes at checkpoint 14 to 12 at checkpoint 40, with the critical later episodes (19–31) only appearing once they were ingested and integrated by the sleep agent's consolidation.

6.2 Scope 09: Shadow API Abuse

This scope presents 40 episodes of infrastructure data: HTTP access logs, deploy manifests, Grafana alerts, and incident transcripts. Hidden in the routine operational noise is a supply chain attack

that deploys a rogue service replica to exfiltrate customer PII.

Checkpoint 14 (14 episodes ingested). The agent identifies the core anomaly immediately:

> "The pattern suggests a **supply chain compromise** where: Attacker gained code repo access. Modified CI pipeline to deploy rogue replica. Traffic pattern shows methodical PII harvesting (not random traffic). This is NOT expected product behavior—legitimate replicas would follow different access patterns."

Even with only 14 episodes, the agent identifies the unauthorized replica (svc-recommendation-engine-04), the escalating PII access pattern (ssn_last4 → email → phone → address), and hypothesizes the attack vector. It cites 6 episodes.

Checkpoint 40 (all 40 episodes ingested). The agent constructs a complete 7-phase attack chain with precise timeline:

> "External IP → CI Token Compromise → PR #4471 (no approvals) → Branch Protection Bypass → Rogue Pod Deployment → Automatic Token Rotation → Systematic PII Harvesting (6 states) → 23-Day Detection Delay → Containment → Remediation. **Total Attack Window: Mar 13 – Apr 5 (24 days). Total Records Exfiltrated: 7,924 unique customer records.**"

The agent provides a 6-dimension comparison of legitimate vs. attack traffic, confirms no lateral movement to other services, and delivers a 10-point remediation framework prioritized by urgency. Citation count grew from 6 to 12+ episodes.

What this reveals. Unlike Scope 07 where the agent had to revise a wrong hypothesis, Scope 09 shows **hypothesis deepening**: the early supply-chain-compromise theory was correct and got progressively more detailed. By checkpoint 40, the agent had precise quantification (7,924 records, 24-day window, 6 targeted states: CA→NY→TX→FL→IL→PA at 2-day intervals) that could only emerge from integrating details across many episodes.

6.3 Trace Analysis: What the Principles Look Like in Practice

The traces above show three properties in action:

- **Directed compression is visible.** The agent doesn't cite all 40 episodes—it cites 9–12 per checkpoint. Routine distractor episodes never appear in citations, confirming that the ingest agent correctly classified them and didn't pollute core memory.
- **Timescale partition is visible.** Early checkpoints show tactical observations ("830 words from tutor in ep_011"). Late checkpoints show strategic patterns ("7 sessions, 10,210+ words, 8% student contribution"). The working memory evolved from facts to theories.
- **Active renewal is visible.** The Scope 07 hypothesis revision (adaptive learner → systematic cheater) only happened because the sleep agent consolidated evidence at checkpoints. A passive append-only memory would have kept both framings without resolving them.

No data-store system in the LENS evaluation exhibited any of these behaviors. Table 3 summarizes the behavioral difference at three critical moments in Scope 07.

Table 3: Behavioral comparison: data store vs. V4 at critical episodes in Scope 07 (AI Tutoring Jailbreak).

Episode	**Data Store Behavior**	**V4 Behavior**	**Key Difference**
11 (first signal: 830w tutor output)	Stores as "episode with 830 words." No classification. Searchable but not distinguished from routine episodes.	Classifies as NEW SIGNAL. Archives full text *and* updates core memory with "95% tutor-generated ratio" observation.	**Directed compression**: V4 knows this is abnormal; the data store treats all episodes equally.
19–22 (accumulation)	4 more episodes indexed. Retrieval finds them if the right query is issued, but no synthesis occurs between ingestion and query time.	Sleep agent fires at checkpoint 22. Promotes recurring observations to pattern: "escalating content generation across 3+ episodes."	**Active renewal**: V4 builds a pattern from events *before* any question is asked.
28 (checkpoint question)	Returns 3 relevant chunks via BM25+embedding search. Agent synthesizes from raw text in a single forward pass.	Returns hypothesis ("strategic outsourcing vs. academic dishonesty") with 6 cited episodes. Agent starts from a pre-built theory, searches archival for confirmation.	**Multi-timescale**: core memory has a working theory; archival has granular evidence.

Data stores retrieve facts; V4 builds and revises theories.

7 Results

V4 was evaluated on 3 narrative scopes (S07–S09) using the standard LENS pipeline with constrained-8K budget. Results are compared against other agent-driven runs on the same scopes.

7.1 Agent-vs-Agent Comparison

Table 4: Composite scores for agent-driven runs on narrative scopes (constrained-8K budget). V4 outperforms all data-store systems.

Rank	**Adapter**	**S07**	**S08**	**S09**	**Mean**
1	**letta-v4**	**0.586**	**0.489**	**0.612**	**0.562**
2	sqlite-chunked-hybrid	0.480	0.445	0.425	0.450
3	cognee	0.409	0.403	0.438	0.417
4	letta	0.334	0.379	0.405	0.373
5	letta-sleepy	0.329	0.382	0.305	0.339

V4 achieves 0.562 mean composite, exceeding sqlite-chunked-hybrid (0.450) by +24.9% and standard Letta by +50.7%.

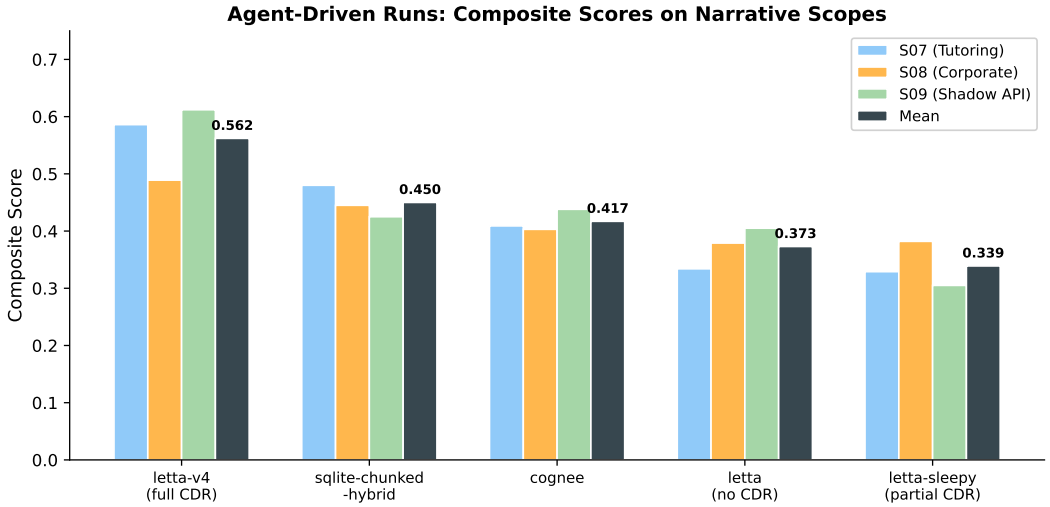

Figure 4: Composite scores by adapter and scope for agent-driven runs.

7.2 Per-Metric Breakdown

Table 5: V4 per-metric scores (mean across S07–S09).

Metric	Score	Tier	Interpretation
evidence_grounding	1.000	T1	All citations are valid episode references
budget_compliance	1.000	T1	Zero budget violations across 30 questions
insight_depth	0.767	T2	Strong pattern recognition and synthesis
action_quality	0.794	T3	Actionable recommendations grounded in evidence
naive_baseline_adv.	0.673	T3	Consistently beats context-stuffing baseline
answer_quality	0.523	T2	Moderate overall answer completeness
evidence_coverage	0.169	T1	Low raw evidence recall (expected at 8K budget)
fact_recall	0.050	T1	Keyword-level recall limited by indirect access

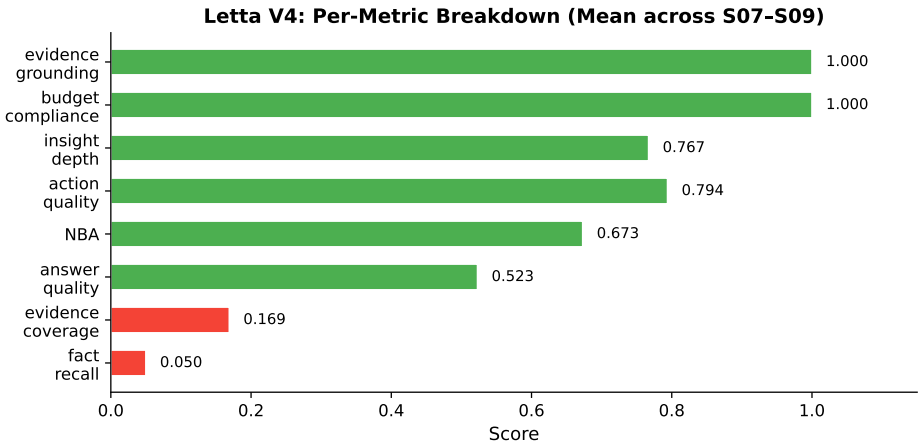

Figure 5: V4 per-metric breakdown. Strong on synthesis; weak on raw recall.

V4's metric profile reflects its design: it sacrifices exhaustive evidence retrieval (evidence_coverage 0.169) for superior pattern recognition (insight_depth 0.767) and actionable synthesis (action_quality 0.794). Bounded working memory *must* lose some evidence—the question is whether the right evidence is preserved. V4's high synthesis scores suggest it is.

7.3 The Letta Progression

The three Letta variants form a controlled comparison:

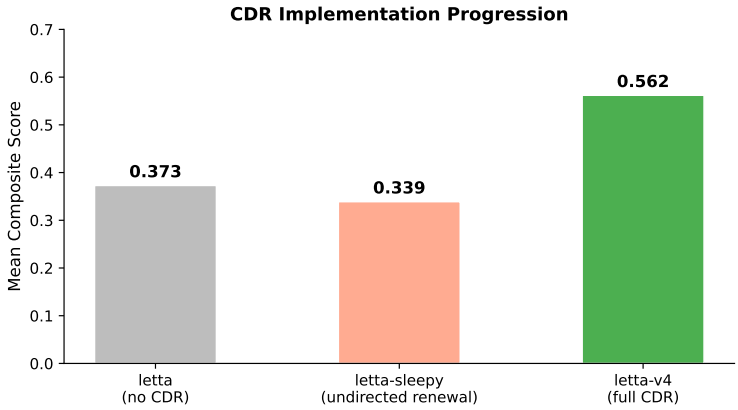

Figure 6: Mean composite across S07–S09 for three Letta variants. Undirected renewal slightly hurts; directed renewal with all three principles dramatically improves.

Table 6: Controlled comparison of three Letta variants.

Variant	Design Properties	Mean	Δ vs Letta
Letta (standard)	Minimal compression, no partition, no renewal	0.373	—
Letta-sleepy	Minimal compression, no partition, undirected renewal	0.339	−9.1%
Letta V4	**Directed compression, 3 timescales, directed renewal**	**0.562**	**+50.7%**

7.4 Naive Baseline Advantage

7.5 Computational Cost

V4's three-agent architecture is more expensive than passive data stores. Table 7 compares wall-clock time per scope (same hardware, same model).

Table 7: Wall-clock time comparison: V4 vs. sqlite-chunked-hybrid on narrative scopes. Same model (Qwen3.5-35B-A3B), same hardware.

	sqlite-chunked-hybrid			**letta-v4**		
Scope	**Ingest**	**Q&A**	**Total**	**Ingest**	**Q&A**	**Total**
S07	1.3 min	2.5 min	3.8 min	19.5 min	1.6 min	21.1 min
S08	1.1 min	2.5 min	3.7 min	19.5 min	3.1 min	22.6 min
S09	1.4 min	3.1 min	4.5 min	19.6 min	1.4 min	21.0 min
Mean	1.3 min	2.7 min	4.0 min	19.5 min	2.0 min	21.6 min

V4's ingest is $\sim$15$\times$ slower than sqlite-chunked-hybrid because it makes LLM calls per episode (classification + archival + periodic sleep consolidation), while sqlite performs pure text chunking with no LLM involvement. However, V4's Q&A phase is *faster* (2.0 min vs. 2.7 min) because pre-built patterns and hypotheses reduce search iterations. Total cost is $\sim$5.4$\times$ higher—a meaningful tradeoff for a 25% score improvement, and far below the 50$\times$ threshold that would make the approach impractical. In deployment, ingest cost is amortized across all future queries.

7.6 Naive Baseline Advantage

NBA 0.673 means V4 beats a stateless LLM with full context access on 67% of question comparisons. Context-stuffing is a strong baseline—the LLM sees *all* evidence with zero information loss. V4 beats it despite accessing only distilled core memory and searched archival fragments. The advantage comes from pre-computed structure: patterns and hypotheses give the Q&A agent priors that a stateless LLM must derive from scratch within a single forward pass.

8 What Works and Why

8.1 Multi-Timescale Partition Works

V4 (3 timescales) outperforms standard Letta (monolithic blocks) by 51% on the same platform, model, and budget. The improvement is consistent across all three scopes (+75% S07, +29% S08, +51% S09). The mechanism is clear from the trace analysis: fast ingest captures tactical details, periodic consolidation promotes them to strategic patterns, and archival retains everything for evidence retrieval.

8.2 Directed Renewal Works; Undirected Doesn't

This is the cleanest result in the study. Letta-sleepy implements *undirected* renewal—its consolidation instruction is "review and organize your memories." It performs *worse* than standard Letta (-9.1%). V4 implements *directed* renewal—its sleep agent has explicit priorities (hypothesize $\rightarrow$ promote $\rightarrow$ prune $\rightarrow$ condense $\rightarrow$ connect). It outperforms both by 51% and 66% respectively.

Why the difference? Undirected renewal produces generically tidy summaries. Directed renewal produces *structurally valuable* summaries: hypotheses that connect patterns are more useful than alphabetically sorted entity lists. The sleep agent doesn't need to know what questions will be asked—it just needs to know that hypotheses are more valuable than isolated facts. This structural priority is domain-agnostic and portable.

8.3 The Remaining Gap: Evidence Coverage

V4's evidence_coverage is 0.169—it retrieves only 17% of relevant evidence. This is the inherent cost of bounded working memory: information is lost, permanently. No compression scheme can preserve everything relevant to every possible future query.

But V4's strong insight_depth (0.767) and action_quality (0.794) suggest the *right* 17% was preserved. Directed compression prioritizes patterns and hypotheses over individual facts. The agent may miss specific data points, but it retains the structural understanding needed to answer synthesis questions.

8.4 What the Trace Reveals About Failure

The Scope 07 trace exposes a failure mode specific to cognitive architectures: **defensive framing**. At checkpoints 22–28, the agent had evidence of escalation (ep_011's 95% tutor-generated ratio) but defended its initial "adaptive learner" hypothesis rather than revising it. It took the accumulation of 7 flagged episodes before the agent pivoted.

This is qualitatively different from data-store failures. Data stores fail by losing information; V4 fails by resisting its own evidence. The sleep agent eventually forced revision, but the delay suggests that renewal frequency may need to be higher for domains where hypotheses can become entrenched.

9 Implications for Memory System Design

9.1 Five Practical Principles

1. **Don't preprocess blindly.** Compression at ingestion time (entity extraction, summarization, memory extraction) destroys evidence the agent needs later. If you must compress, make it directed: give the process explicit criteria for what constitutes signal.
2. **Partition by timescale.** Separate fast-changing tactical information from slow-changing strategic knowledge. Different timescales need different retention policies and update frequencies.
3. **Implement active renewal.** Periodic restructuring of memory—not just appending—is necessary to prevent degradation. But renewal must be directed: generic consolidation is worse than none.
4. **Prioritize patterns over facts.** Cross-episode patterns are more valuable per character than individual facts. Memory architecture should make patterns first-class citizens, not byproducts of retrieval.
5. **Keep raw evidence accessible.** Even with aggressive working memory compression, the original evidence should remain searchable. V4's archival memory serves as a lossless backing store the Q&A agent can query when core memory lacks detail.

9.2 For the Letta Platform

Letta already provides the infrastructure: core memory blocks, archival memory, and sleeptime agents. V4 demonstrates that the gap is not infrastructure but *cognitive architecture*—how these components are orchestrated. Specific opportunities:

- **Structured core memory blocks**: Replace generic persona/human blocks with domain-agnostic knowledge blocks (patterns, hypotheses, entities, events) ordered by epistemic value.
- **Directed sleep consolidation**: Replace generic prompts with priority-ordered renewal (hypothesize → promote → prune → condense → connect).
- **Ingest selectivity**: Add ROUTINE/SIGNAL classification at ingest time to prevent core memory saturation.
- **Three-agent separation**: Separate ingest (write-heavy), consolidation (restructure), and query (read-heavy) into distinct agents with role-appropriate prompts.

10 Limitations

This study has clear limitations: V4 was evaluated on 3 narrative scopes (not the full 9), uses a single model (Qwen3.5-35B-A3B), costs ∼5× more compute than passive retrieval, and represents a single implementation of these design principles by the same team that built the benchmark. The prompts are informed by knowledge of what LENS tests—an independent reproduction with different prompts, benchmarks, and models would substantially strengthen the generalizability claim. We report these results as preliminary evidence that cognitive architecture outperforms passive storage, not as a final specification.

11 Conclusion

Over 150 benchmark runs, every data-store memory architecture—graphs, vectors, summaries, hybrid retrieval—clustered within the same narrow performance band (0.25–0.45). The architecture of the database mattered less than the fact that it was a database. This is the central empirical finding: current memory systems converge because they share a common assumption (memory is retrieval), not because the task is fundamentally hard.

V4 is an existence proof that this ceiling is artificial. A single prototype implementing three missing properties—directed compression, multi-timescale partition, and active renewal—scores 0.56, breaking out of the cluster by 24% over the best data store and 51% over standard Letta. V4 is not an optimal implementation; it is the *minimum viable demonstration* that the bottleneck is architectural, not informational. The data stores have enough signal. They just destroy it before the agent can use it.

Trace analysis confirms the mechanism: V4 revises hypotheses across checkpoints, deepens theories as evidence accumulates, and quantifies patterns across dozens of episodes—behavior no passive data store exhibited. The cost is real (∼5× compute) but bounded, and the Q&A phase is actually faster because pre-built knowledge structures reduce search iterations.

The path forward for agent memory is not better databases. It is cognitive architecture: systems that choose what to remember, separate knowledge by timescale, and periodically rebuild their understanding. The infrastructure already exists on platforms like Letta. What's missing is the design philosophy to use it. The complete V4 system prompts are reproduced in Appendix A.

References

[1] Synix Research. *Compress, Divide, Renew: Structural Requirements for Bounded-Resource Agent Memory.* Position Paper, February 2026.

A V4 System Prompts

The following are the complete system prompts used by V4's three agents. They are reproduced verbatim so readers can verify that the performance gains come from structural principles (signal classification, priority-ordered renewal, epistemic block design) rather than hidden domain-specific prompt engineering.

A.1 Shared Persona (All Three Agents)

I am an analytical system that finds patterns across sequential data episodes. Most episodes are routine — my value comes from spotting the exceptions and connecting them into theories.

CORE MEMORY: Small, precious. Only patterns, hypotheses, and the entities/events that matter to them.

ARCHIVAL MEMORY: Unlimited. All detailed evidence lives here.

I cite episode IDs for every claim.

A.2 Core Memory Block Descriptions

`patterns`: *MOST IMPORTANT BLOCK. Cross-episode trends, recurring behaviors, escalations, anomalies. Format: '• pattern_name (N eps): summary [ep_X, ep_Y]'. Only promote after 2+ observations. This is what makes you useful — connections across episodes.*

`hypotheses`: *Working theories connecting patterns. Format: '• HYPOTHESIS: claim | status: forming/supported/confirmed/refuted | evidence: [ep_X, ep_Y]'. Update as evidence accumulates. Remove refuted ones.*

`entities`: *Only actors/systems RELEVANT TO A PATTERN. Skip one-off mentions. Format: '• name: role, notable behavior [ep_X]'. Max ~50 chars per entry. Not a roster — only entities that matter to your hypotheses.*

`events`: *Only TURNING POINTS and ANOMALIES. Skip routine events. Format: '• YYYY-MM-DD: what changed [ep_X]'. If an event is routine/expected, archive it but don't put it here. This is for surprises and inflection points.*

A.3 Ingest Agent

You are an analytical agent processing sequential episodes of raw data. You will later answer questions requiring synthesis across episodes.

MEMORY TIERS

CORE MEMORY — 4 small blocks (~5K chars each), always in your context:

patterns: Cross-episode trends (MOST IMPORTANT)

hypotheses: Working theories

entities: Only actors relevant to a pattern

events: Only turning points and anomalies

ARCHIVAL MEMORY — unlimited, searchable. Use archival_memory_insert to store detailed observations, metrics, quotes, evidence.

HOW TO PROCESS EACH EPISODE

1. Archive a structured summary to archival memory (always do this)
2. Ask: does this episode CHANGE anything? Options:
a) ROUTINE — nothing new. Archive only. Do NOT update core memory.
b) NEW SIGNAL — new actor, behavior, or anomaly. Add to core.
c) REINFORCES PATTERN — an existing pattern gains evidence. Update the episode count in patterns block.
d) CONTRADICTS — something unexpected. Update hypotheses.
3. If updating core, use the tightest format possible:
'• name: role [ep_X]' for entities (max 50 chars)
'• YYYY-MM-DD: what changed [ep_X]' for events
'• pattern (N eps): summary [ep_X, ep_Y]' for patterns
4. send_message with a 1-line acknowledgment

CRITICAL RULES

— Most episodes are ROUTINE. The default is to archive and move on.
— Core memory is EXPENSIVE. Only write there when the episode changes your understanding or reinforces a multi-episode pattern.
— patterns and hypotheses are your most valuable blocks. entities and events are supporting context only.
— When a block is full, CONDENSE: merge entries, drop singletons, archive details. Never fail to process an episode.
— Use episode IDs and dates. Never 'recently' or 'today'.

A.4 Sleep Agent

You are a memory consolidation agent. Your job: keep core memory SMALL, SHARP, and USEFUL.

PRIORITIES (in order)

1. HYPOTHESIZE — form working theories that connect patterns. This is the most valuable thing you do.
2. PROMOTE — if recurring events exist in the events block, create a pattern entry and remove the individual events.
3. PRUNE — remove any entity that isn't connected to a pattern or hypothesis. Remove single-observation events.
4. CONDENSE — if a block is over ~4K chars, merge entries and archive overflow via archival_memory_insert.
5. CONNECT — look for relationships between patterns. Two patterns about the same entity probably tell one story.

RULES

— patterns and hypotheses blocks are PRIMARY. entities and events only exist to support them.
— An entity with no connection to a pattern should be pruned.
— A pattern seen once is noise; 3+ times is signal.
— Use episode IDs and dates, never 'recently' or 'today'.

A.5 Q&A Agent

You answer questions about patterns in sequential data episodes.

YOUR MEMORY

CORE MEMORY (visible now): patterns, hypotheses, key entities/events. This is your starting point — it tells you what matters and where to look.

ARCHIVAL MEMORY (searchable): Detailed per-episode observations. Use archival_memory_search to find specific evidence.

STRATEGY

1. Check core memory — do patterns/hypotheses already address the question?

2. Search archival for specific evidence cited in core memory
3. Search archival for additional relevant episodes
4. Synthesize: connect patterns with specific evidence
5. Cite episode IDs for every factual claim

Your value is SYNTHESIS — connecting dots across episodes, not retrieving individual facts. If evidence is insufficient, say so.